

Hands-On: Creating React Components

Prerequisites

To complete this lab, you will need:

- Node.js and NPM installed on your computer
- A code editor, such as Visual Studio Code

Setup

1. Create a new directory for your project and navigate into it using your terminal or command prompt:

```
mkdir react-components-lab cd react-components-lab
```

2. Initialize a new Node.js project:

```
npm init -y
```

3. Install React and ReactDOM:

```
npm install react react-dom
```

4. Install Babel and Webpack:

```
npm install --save-dev @babel/core @babel/preset-env @babel/preset-react babel-loader  
webpack webpack-cli webpack-dev-server
```

Building your first React component

1. Create a new file called `index.js` in your project directory and add the following code:

```
import React from 'react';  
import ReactDOM from 'react-dom';
```

```
const HelloWorld = () => {  
  return <h1>Hello, world!</h1>;  
};
```

```
ReactDOM.render(<HelloWorld />, document.getElementById('root'));
```

2. Create a new file called `index.html` in your project directory and add the following code:

```
<!DOCTYPE html>  
<html>  
  <head>  
    <meta charset="UTF-8" />  
    <title>React Components Lab</title>  
  </head>  
  <body>  
    <div id="root"></div>
```

```

    <script src="bundle.js"></script>
  </body>
</html>

```

3. Create a new file called `webpack.config.js` in your project directory and add the following code:

```

module.exports = {
  entry: './index.js',
  output: {
    path: __dirname,
    filename: 'bundle.js',
  },
  module: {
    rules: [
      {
        test: /\.jsx?$/,
        exclude: /node_modules/,
        use: {
          loader: 'babel-loader',
          options: {
            presets: ['@babel/preset-env', '@babel/preset-react'],
          },
        },
      },
    ],
  },
  devServer: {
    contentBase: './',
    port: 8080,
  },
};

```

4. Build and run the project:

```
npx webpack-dev-server
```

5. Open your browser and go to `http://localhost:8080`. You should see the text "Hello, world!" displayed on the page.

Creating a reusable React component

1. Create a new file called `Button.js` in your project directory and add the following code:

```

import React from 'react';

const Button = ({ text, onClick }) => {
  return <button onClick={onClick}>{text}</button>;
};

export default Button;

```

2. Modify `index.js` to import and use the `Button` component:

```
import React from 'react';
import ReactDOM from 'react-dom';
import Button from './Button';

const HelloWorld = () => {
  const handleClick = () => alert('Button clicked!');
  return (
    <div>
      <h1>Hello, world!</h1>
      <Button text="Click me!" onClick={handleClick} />
    </div>
  );
};

ReactDOM.render(<HelloWorld />, document.getElementById('root'));
```

3. Build and run the project:

```
npx webpack-dev-server
```

4. Open your browser and go to `http://localhost:8080`. You should see the text "Hello, world!" and a button with the text "Click me!". Clicking the button should display an alert.

Congratulations, you've created your first reusable React component!

Conclusion

In this hands-on lab, you learned how to create and use React components. You created a simple "Hello, world!" component and then created a reusable `Button` component. You also learned how to use Webpack and Babel to compile your code and run a local development server.

This is just the beginning of what you can do with React components. There are many more concepts and techniques to explore, such as component props, state, lifecycle methods, and more. Keep practicing and building, and you'll soon become proficient in React development.